

Relatório da Simulação do Sistema ESP32 Integrado ao Django

1. Introdução

Este relatório descreve o processo de simulação de um sistema de automação residencial baseado em ESP32, sensores digitais e analógicos, atuadores (LEDs representando atuadores digitais), e comunicação com um servidor Django. O objetivo da simulação é validar a operação do sistema, incluindo:

- Aquisição de dados dos sensores
- Processamento de informações
- Controle de atuadores
- Comunicação com o servidor
- Gerenciamento centralizado com Django.

A simulação permite testar a lógica de controle e integração com a API sem a necessidade de hardware físico completo, utilizando o ambiente Wokwi para ESP32, e o python anywhere para host do servidor django.

2. Configuração Inicial da Simulação

2.1 Ambiente de Simulação

A simulação foi realizada utilizando o **Wokwi ESP32 Simulator**, que permite emular:

- Entradas digitais (botões de gás, fumaça e presença)
- Entradas analógicas (potenciômetros simulando umidade e temperatura)
- Saídas digitais (LEDs simulando atuadores digitais)

2.2 Componentes e Endereçamento

Os dispositivos foram mapeados da seguinte forma:

Componente	Pin ESP32	Função
Cortina	5	LED/Atuador

Fechadura	15	LED
Luz Sala	2	LED
Botão Gás	21	Entrada digital
Botão Fumaça	22	Entrada digital
Botão Presença	23	Entrada digital
Irrigação	18	LED/Atuador
Potenciômetro Umidade	32	Entrada analógica
Potenciômetro Temperatura	33	Entrada analógica

2.3 Conexão com Django

A API do Django foi configurada com endpoints REST para cada dispositivo:

- **GET /device/{id}/read/** → Retorna estado atual do dispositivo
- **POST /device/{id}/write/** → Atualiza estado do dispositivo

O ESP32 se conecta via WiFi ao servidor, enviando dados dos sensores e recebendo instruções de controle.

3. Processo de Simulação

3.1 Inicialização

1. O ESP32 conecta-se à rede WiFi simulada (**Wokwi-GUEST**).
2. Inicializa todos os pinos de entrada e saída.
3. Prepara os LEDs de erro para indicar falhas de comunicação.

3.2 Aquisição de Dados

- Sensores digitais são lidos continuamente para detectar eventos.
- Sensores analógicos de temperatura e umidade são lidos e mapeados para uma escala de 0 a 10.

3.3 Processamento e Controle

O sistema aplica regras de controle simples:

Cortinas

- Ligam quando o servidor Django retorna estado 1.

Irrigação

- Liga apenas quando uma das condições de umidade e temperatura for atendida:
 1. Umidade baixa e temperatura alta
 2. Umidade baixa e temperatura baixa
 3. Umidade média e temperatura alta
- A lógica é executada a cada leitura de sensores, garantindo resposta imediata.

3.4 Comunicação com Django

- Valores de sensores são enviados via POST para os endpoints correspondentes.
- Estados de dispositivos são lidos via GET.
- LEDs de erro acendem caso haja falha na comunicação.

4. Análise dos Resultados

Durante a simulação, percebeu-se um grande atraso na resposta do sistema físico, que pode ser explicada pela conexão interna do wokwi ser demorada para responder as requisições HTTP e o programa funcionar de forma síncrona, porém foram observados os seguintes comportamentos positivos:

1. Respostas do ESP32 aos sensores

- Os botões simulados mudam corretamente os estados e o ESP32 envia os dados para Django.

2. Acionamento de cortinas

- Ao receber estado 1 do servidor, a cortina liga corretamente.

3. Acionamento da irrigação

- A irrigação é acionada apenas quando as condições de temperatura e umidade são atendidas.
- O LED da irrigação permanece ligado enquanto o sinal de acionamento estiver ativo no ESP32.

4. Falhas de comunicação

- LEDs de erro acendem corretamente quando há falha no POST ou GET, permitindo fácil identificação de problemas.

5. Validação da Simulação

A validação do sistema foi realizada comparando os sinais de entrada com as ações de saída:

- Sensores digitais** → Acionamento correto de LEDs e envio correto para Django.
- Sensores analógicos** → Valores mapeados corretamente e lógica de irrigação funcionando conforme as regras definidas.
- Cortinas** → LEDs apontaram funcionamento correto.
- Comunicação com Django** → Todos os dados enviados e recebidos correspondem ao esperado, confirmando integração bem-sucedida, porém com grande atraso,

A simulação mostrou que o sistema atende aos requisitos de controle e monitoramento, permitindo testes completos sem hardware físico.

6. Conclusão

A simulação do ESP32 integrado ao Django demonstrou que o sistema de automação:

- Lê sensores digitais e analógicos corretamente
- Controla LEDs e cortinas de forma precisa e temporizada
- Executa regras de lógica de irrigação baseadas em umidade e temperatura
- Mantém comunicação confiável com o servidor Django

Este relatório fornece documentação detalhada de todo o processo de simulação, servindo como referência para implementação em hardware real, testes futuros e validação de novas funcionalidades.